

NESTO

CODE AUDIT 2

Unified Module Logic, Domain Ownership and Cross-Module Interaction

Engineering audit and target architecture guidance for the Full Stack Development Team

Scope: module logic, source-of-truth ownership, authorization, events, workflows, finance, dashboards and auditability

Prepared from the repository review and PRD_1 through PRD_14

03 August 2026

1. Executive Assessment

This audit focuses on the logic behind Nesto modules and how they should behave as one operating system rather than a collection of separate applications.

Overall judgment

Nesto has strong domain depth and an exceptional product vision. The main engineering risk is not the number of modules. The risk is allowing each module to create its own permissions, statuses, documents, comments, financial totals and audit behavior. The platform should keep its ambition, but all modules must use shared control mechanisms and explicit domain ownership.

Area	Current risk	Required direction
Module boundaries	Overlapping responsibility and duplicated records	One owner module for every business fact
Permissions	Role checks can differ by page or server action	One policy engine using live company and record context
Cross-module actions	Direct table writes create hidden coupling	Publish events and let owning modules react
Finance	Manual totals can disagree with payment records	Finance-approved values plus reconciliation controls
Dashboards	Sensitive values may be queried before being hidden	Permission-safe read models
Audit	Separate activity logic can become inconsistent	One immutable activity and audit stream

Primary engineering principle: Every record has one source of truth. Every action has one authorization policy. Every cross-module consequence uses an event. Every important change produces an audit entry.

2. Core Logic Weaknesses

These weaknesses should be corrected before module count and customization increase further.

2.1 Duplicated business truth

A Client page, Contractor page, Project page and Finance page may each be tempted to store their own contract or payment totals. This creates disagreement and difficult corrections. Profiles must aggregate data from the owning modules instead of storing copied values.

2.2 Module-specific authorization

A page may hide a button while a backend action remains callable, or one module may define Admin differently from another. Authorization must be evaluated in one central service for every request and record.

2.3 Direct cross-module database writes

When Contract code directly edits Finance, Project or Document tables, releases become fragile and ownership becomes unclear. Cross-module effects should be triggered through durable domain events.

2.4 Free-form lifecycle changes

Allowing statuses to be edited as ordinary fields permits invalid states. Every controlled record needs an explicit state machine and approved transition list.

2.5 Summary values treated as editable records

Totals such as paid, remaining, contract exposure and client receivables should be derived read models or Finance-approved fields with independent reconciliation. Users must not directly edit consolidated totals.

2.6 Oversized orchestration services

Large services accumulate creation, approvals, inspections, escalations, notifications and queries. They should remain within one domain but be separated by responsibility.

Critical risk if unchanged

As the platform grows, a single contract or payment update could produce conflicting values across several pages. Security fixes would also need to be repeated in many modules. This increases production incidents, audit disputes and development cost.

3. One Canonical Action Pipeline

All important commands must pass through the same sequence, regardless of module or interface.

Step	Responsibility	Mandatory output
1. Identity	Resolve authenticated user and active session	Actor ID and session context
2. Company context	Resolve tenant, active company, group/JV context and active role	Verified company scope
3. Authorization	Evaluate action against module, record and relationship policy	Allow or deny decision
4. Validation	Validate input, state transition and linked-record ownership	Validated command
5. Transaction	Write only through the owning domain service	Atomic database result
6. Audit	Record actor, action, before/after, company and source	Immutable audit entry
7. Event	Publish a durable domain event after successful commit	Outbox event
8. Reactions	Create notifications, read-model updates and linked tasks	Idempotent downstream effects

Non-negotiable backend rule

No page, route handler, server action, cron job or import script may write directly to business tables unless it uses the same command service, authorization policy, transaction boundary and audit pipeline.

4. Domain Ownership Matrix

Each business fact must have one authoritative module. Other modules may reference or display it, but must not create a second version.

Owning module	Authoritative records	Other modules may do
Projects	Project identity, members, project status, progress and milestones	Reference project; display permitted summaries
Contracts	Agreement, parties, revisions, approval status and legal lifecycle	Link tasks, payments, units and documents
Finance	Approved values, invoices, payments, balances and reconciliation	Display authorized financial read models
Clients	Client identity, contacts, representatives and relationships	Link contracts, units and receivables
Contractors	Contractor identity, qualifications and company relationship	Link contracts, projects, evaluations and payables
Units / Architecture	Unit identity, floor plans, design state and approved unit comments	Link buyer, project and contract
Tasks	Assignments, deadlines, approvals, completion and reopen logic	Create tasks through commands or events
Documents	Files, revisions, official status, retention and access	Link documents; never duplicate file versions
HR	Employees, leave, attendance and private HR records	Expose only authorized availability/read models
Platform Administration	Tenants, features, support access and platform governance	Never own company operational data

Profile rule: Client and Contractor pages are composite views. They pull identity, contracts, units, payments, projects, documents and tasks from their owning modules. A profile is not a parallel database.

5. Cross-Module Interaction Through Domain Events

Modules should communicate through durable events rather than hidden direct writes.

Event	Published by	Permitted reactions
ContractApproved	Contracts	Finance creates financial structure; Documents locks approved revision; Project displays active contract
PaymentRecorded	Finance	Balances recalculate; client/contractor summary updates; permitted notifications are created
ContractCommentTaggedUser	Comments / Contracts	Task module creates one linked task for the tagged user
TaskCompleted	Tasks	Source comment or record displays completion status; reviewer receives notification
LeaveApproved	HR	Personal calendar and Team Availability read models update; no task reassignment
CompanyFeatureChanged	Platform Admin	Tenant feature cache refreshes; oversight log records scope and version

Events must be written to an outbox table in the same database transaction as the source change. A background worker publishes and processes them. Every handler must be idempotent so retries do not create duplicate tasks, payments, notifications or approvals.

Example: Finance tags an Architect

The contract comment remains owned by the Contract/Comment domain. The Task domain receives an event and creates one task linked to the client, contract, purchased unit and original comment. The Architect completes the task. Finance reviews and formally closes it. Both records display the same lifecycle without duplicating each other.

6. Central Authorization and Data Isolation

Module entry permissions and exact-record permissions must be evaluated separately.

Policy input	Examples
Actor	User ID, active role, department, employment state
Company scope	Tenant, active company, parent/child company, JV scope
Resource	Record type, record ID, owning company and sensitivity
Relationship	Project member, contract participant, department access, record owner
Temporary authority	Delegation, Platform Admin support access, standing authorization
Requested action	View, create, edit, approve, delete, restore, export, impersonate

Every linked ID supplied by the frontend must be resolved and verified against the active tenant and company. The backend must reject cross-tenant or cross-company relationships even when the user knows a valid record ID.

Security requirement

Restricted values must never be queried and sent to the client merely because the interface hides them. Authorization must shape the database query or read model itself.

7. Controlled Record Lifecycles

Important statuses must be governed by transition services, not freely edited fields.

Record	Example lifecycle	Control
Contract	Draft -> Under Review -> Approved -> Active -> Completed / Terminated	Only approved actors may transition; revisions remain traceable
Task	Draft -> Assigned -> In Progress -> Ready for Review -> Completed	Reviewer rules enforced on backend; reopen creates audit entry
Payment	Draft -> Confirmed -> Posted -> Reversed	Posted amounts immutable; reversal creates counter-record
Document	Draft -> Under Review -> Official -> Superseded / Archived	Official revision cannot be silently replaced
Red Flag	Open -> Under Review -> Closed	Closed flag cannot reopen; new flag required

Each transition service should define: allowed source states, required permission, mandatory fields, transaction writes, audit event, domain event, notifications and idempotency key.

8. Financial Truth and Reconciliation

Preserve the confirmed Finance authority while making inconsistencies visible and technically manageable.

- Finance manually enters and approves contract values.
- Every received or paid amount is stored as an individual payment record with date, amount, reference, method and proof document.
- Finance may enter approved paid and remaining figures where required by the PRDs.
- The system independently calculates expected paid and remaining totals from payment records.
- Differences create a reconciliation warning; the platform does not silently overwrite Finance-approved values.
- Consolidated client and contractor totals are read models, not directly editable records.
- Use database Decimal types for money and store currency explicitly.
- Posted financial records are never hard-deleted; corrections use reversal or adjustment records.

Displayed field	Source	Editable?
Approved contract value	Finance-approved contract financial record	Finance only
Recorded payments	Individual payment records	Controlled create/reverse
System-calculated paid total	Sum of valid posted payments	No
Finance-approved paid total	Manual Finance approval where required	Finance only
System-calculated remaining	Approved value minus posted payments	No
Reconciliation status	Comparison of approved and calculated values	No

9. Universal Linking, Read Models and Activity Timeline

Universal linking: Tasks, comments, documents, meetings and red flags should link to supported records through a controlled reference layer. Each reference must include tenant ID, company ID, target type and target ID, and must be validated before creation.

Linkable source	Typical targets
Task	Project, contract, client, contractor, unit, payment, invoice, document or employee
Comment	Contract, task, unit, document, client or contractor
Document	Project, contract, unit, payment, client, contractor or employee
Meeting / MoM	Project, task, contractor work package or document review
Red Flag	Any controlled business record the actor may access

Permission-safe read models: Dashboards and profile summaries should read from dedicated projections such as Executive Project Summary, Client Account Summary, Contractor Payable Summary and Personal Work Summary. Each projection must contain only fields visible to its audience.

One activity timeline: Every major page should use a common timeline built from audit and domain events. It should show creation, edits, approvals, comments, revisions, payments, tasks, status changes and Platform Admin support actions according to the viewer's permissions.

10. Internal Service Decomposition

Keep modules unified from the user perspective while separating internal responsibilities for maintainability.

Domain	Recommended internal services
Tasks	Creation, assignments, approvals, inspections, escalations, completion/reopen, contractor work packages, queries and events
Contracts	Drafting, participants, revisions, approvals, lifecycle, comments, access and events
Finance	Contract financials, invoices, payment allocation, posting, reconciliation, summaries and events
Documents	Upload, storage, revision control, official status, permissions, retention and preview
Platform Admin	Tenant lifecycle, features, releases, support access, recovery, oversight and rollback

Services within the same domain may share repositories and transactions. Different domains must communicate through public commands, queries or events rather than importing private implementation files.

Interface consistency rule

Role-specific themes and navigation may differ substantially, but the underlying commands, statuses, permissions and source records must remain identical. A different interface must not become a different business system.

11. Implementation Priority

The following sequence minimizes risk while allowing feature development to continue.

Priority	Work package	Definition of done
P0	Central policy engine and active company context	Every protected command and query uses live record-level authorization
P0	Tenant/company ownership validation	Every foreign key and polymorphic link is checked against active scope
P0	Money conversion to Decimal	All monetary fields, calculations and tests use Decimal and currency
P1	Command pipeline and audit contract	Shared command wrapper covers validation, transaction, audit and event outbox
P1	Domain ownership cleanup	Duplicated totals and records are replaced by references/read models
P1	Lifecycle transition services	Controlled records reject invalid transitions server-side
P1	Event outbox and idempotent handlers	Cross-module effects retry safely without duplicates
P2	Permission-safe dashboards	Sensitive values are excluded at query level
P2	Universal activity timeline	Major records use one filtered audit/event timeline
P2	Service decomposition	Oversized modules are split internally without duplicating domains

Recommended delivery method

Implement these changes incrementally behind automated tests. Do not rewrite the whole platform at once. Select one vertical workflow - for example Contract Approved -> Finance Structure -> Payment -> Contractor Profile - and use it as the reference implementation for all other modules.

12. Engineering Acceptance Checklist

A module is not complete until all applicable controls below are satisfied.

- The record has one documented owning domain.
- All commands resolve active tenant and company from trusted server context.
- Every linked record is verified to belong to the permitted scope.
- The central policy engine authorizes the exact action and record.
- The command uses a database transaction where multiple writes are required.
- The command creates a structured audit entry with actor and before/after context.
- Cross-module consequences are emitted through an outbox event.
- Event handlers are idempotent and safe to retry.
- Lifecycle transitions are enforced on the backend.
- Restricted fields are excluded from unauthorized queries and read models.
- Financial values use Decimal and reconciliation tests.
- Duplicate submission tests prove that repeated requests do not duplicate records.
- Unit, integration and end-to-end tests cover allowed and denied paths.
- The requirement is traceable to its PRD decision and acceptance criteria.

Final directive to the Full Stack Team

Do not reduce the product vision. Reduce architectural inconsistency. Nesto should remain broad and exceptional for construction and real-estate operations, while presenting one coherent logic: one source of truth, one policy system, one workflow discipline, one audit history and controlled communication between modules.

Audit status: Engineering guidance for implementation planning. This document is not a runtime security certification and should be validated against the current branch during implementation.